

claude-windows / 188dd06f-9c99-487c-aa7d-b0bee83c629c

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\188dd06f-9c99-487c-aa7d-b0bee83c629c\subagents\workflows\wf\_416cc0e5-a35\agent-a7b3f0ab98df2ba0e.jsonl`
- SHA-256: `cc289790a88f9f9502c7e8fcca7ad312d2c2e2ab15a2cbc63ebe79b874e560a6`
- Source modified: `2026-06-22T09:46:44+00:00`
- Imported at: `2026-07-05T16:48:28+00:00`
- project: `wf\_416cc0e5-a35`
- session_id: `188dd06f-9c99-487c-aa7d-b0bee83c629c`

Transcript

1. user (2026-06-22T09:45:35.961Z)

Review the NIGHTLY REEL-COUNT engine-regression change under C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-e2e (a NEW, additive PR). It runs the configured WASB pipeline on a GCP GPU VM nightly, asserts reel-count (==) + quality metrics + cost, emails a report, self-deletes. Files: scripts/nightly_reelcount.py, scripts/nightly_email.py, deploy/gcp/nightly_regression/run_on_vm.sh, deploy/gcp/nightly_regression/launch.sh, deploy/gcp/nightly_regression/register_scheduler.sh, deploy/gcp/nightly_regression/Dockerfile.launcher, eval_baselines/reelcount_manifest.json. Read them under that absolute path. Cite file:line. Report only REAL defects (correctness/safety/security that would bite in production or a regression that escapes), not style. Return raw structured findings.

ADVERSARIALLY VERIFY this finding ? confirm with concrete code evidence under C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-e2e or REFUTE it (default real=false if the evidence isn't clearly there). Finding:

```
{
  "title": "Missing SMTP SSL/TLS context in email send ? potential MiTM attack",
  "file": "scripts/nightly_email.py",
  "line": "86-87",
  "severity": "high",
  "detail": "smtplib.SMTP() is called without an ssl_context parameter. On line 87, starttls() is called to upgrade the connection, but no explicit SSL context is provided. This uses Python's default context, which may be overridable or vulnerable to downgrade attacks depending on the environment. Best practice is to explicitly create a secure context with ssl.create_default_context() and pass it to SMTP(..., context=context).",
  "fix": "Use ssl.create_default_context() to create an explicit SSL context and pass it to SMTP constructor: `import ssl; context = ssl.create_default_context(); s = smtplib.SMTP(host, port, timeout=60, context=context); s.starttls()` (the context parameter replaces the need for separate starttls for security)."
```

2. assistant (2026-06-22T09:45:37.883Z)

I'll analyze the nightly regression change by reading the files and verifying the finding about SMTP SSL/TLS context.

3. user (2026-06-22T09:45:38.904Z)

```
1      """Email the nightly regression report ? stdlib SMTP, creds from env or GCP Secret
Manager.
2
3      Used by ``scripts/nightly_reelcount.py --email`` (and reusable by any nightly).
Deliberately
4      dependency-light (stdlib ``smtplib`` + ``email.message``) and graceful: if no SMTP
password can
```

```

5         be resolved it logs + returns ``False`` (the report is still written/printed ? a broken
mailer must
6         never mask a regression).
7
8         Credential resolution (first hit wins), so it works on a GCP VM, locally, or in tests:
9         ? Recipient: ``--email-to`` arg ? ``NIGHTLY_EMAIL_TO`` ? ``avi.dullu@gmail.com``
10        ? SMTP host: ``NIGHTLY_SMTP_HOST`` (default ``smtp.gmail.com``) . port
``NIGHTLY_SMTP_PORT`` (587, STARTTLS)
11        ? SMTP user: ``NIGHTLY_SMTP_USER`` (default = recipient)
12        ? SMTP pass: ``NIGHTLY_SMTP_PASS`` ? Secret Manager ``NIGHTLY_SMTP_SECRET``
13                (`projects/<p>/secrets/<name>/versions/latest`; via the google-cloud-
secret-manager
14                lib if installed, else the ``gcloud`` CLI). A Gmail app password is
the simplest.
15
16        Set the secret once (owner): echo -n '<app-password>' | gcloud secrets create nightly-
smtp-pass --data-file=-
17        and on the VM: export NIGHTLY_SMTP_SECRET=projects/<proj>/secrets/nightly-smtp-
pass/versions/latest
18                export NIGHTLY_SMTP_USER=<your-gmail>
NIGHTLY_EMAIL_TO=avi.dullu@gmail.com
19        ""
20        from __future__ import annotations
21
22        import logging
23        import os
24        from email.message import EmailMessage
25        from typing import Optional
26
27        log = logging.getLogger("nightly_email")
28
29        DEFAULT_TO = "avi.dullu@gmail.com"
30        DEFAULT_HOST = "smtp.gmail.com"
31        DEFAULT_PORT = 587
32
33
34        def build_message(subject: str, body_md: str, from_addr: str, to_addr: str) ->
EmailMessage:
35        """A text/plain email carrying the markdown report (renders fine in any client).
Pure ? testable."""
36        msg = EmailMessage()
37        msg["Subject"] = subject
38        msg["From"] = from_addr
39        msg["To"] = to_addr
40        msg.set_content(body_md)
41        return msg
42
43
44        def resolve_secret(env: Optional[dict] = None) -> Optional[str]:
45        """Resolve the SMTP password from env or GCP Secret Manager. Returns None if
unresolvable."""
46        env = env if env is not None else os.environ
47        direct = env.get("NIGHTLY_SMTP_PASS")
48        if direct:
49            return direct
50        secret_ref = env.get("NIGHTLY_SMTP_SECRET")
51        if not secret_ref:
52            return None
53        # Prefer the client lib; fall back to the gcloud CLI (no extra dependency required).
54        try:
55            from google.cloud import secretmanager # type: ignore
56            client = secretmanager.SecretManagerServiceClient()
57            return
client.access_secret_version(name=secret_ref).payload.data.decode("utf-8")
58        except Exception: # noqa: BLE001 ? lib absent or auth issue ? try the CLI

```

```

59         pass
60     try:
61         import subprocess
62         name = secret_ref.split("/secrets/", 1)[1].split("/", 1)[0] if "/secrets/" in
secret_ref else secret_ref
63         out = subprocess.run(["gcloud", "secrets", "versions", "access", "latest", "--
secret", name],
64                               capture_output=True, text=True, timeout=30)
65         return out.stdout.strip() if out.returncode == 0 and out.stdout.strip() else
None
66     except Exception as e: # noqa: BLE001
67         log.warning("secret-manager resolution failed: %s", e)
68         return None
69
70
71     def send_report(subject: str, body_md: str, to: Optional[str] = None,
72                   env: Optional[dict] = None) -> bool:
73         """Send the report. Returns True if sent, False if skipped (no creds) ? never raises
on missing creds."""
74         env = env if env is not None else os.environ
75         to_addr = to or env.get("NIGHTLY_EMAIL_TO") or DEFAULT_TO
76         host = env.get("NIGHTLY_SMTP_HOST", DEFAULT_HOST)
77         port = int(env.get("NIGHTLY_SMTP_PORT", DEFAULT_PORT))
78         user = env.get("NIGHTLY_SMTP_USER") or to_addr
79         password = resolve_secret(env)
80         if not password:
81             log.warning("no SMTP password (set NIGHTLY_SMTP_PASS or NIGHTLY_SMTP_SECRET) ?
email skipped.")
82             return False
83
84         import smtplib
85         msg = build_message(subject, body_md, from_addr=user, to_addr=to_addr)
86         with smtplib.SMTP(host, port, timeout=60) as s:
87             s.starttls()
88             s.login(user, password)
89             s.send_message(msg)
90         log.info("emailed report to %s", to_addr)
91         return True
92

```

4. user (2026-06-22T09:45:39.922Z)

```

1     """Nightly END-TO-END engine regression ? reel-count + quality + cost (the GCP-VM
nightly).
2
3     Runs the **configured + checkpointed** pipeline (the real WASB GPU detector, as a
deployment serves
4     it) end-to-end on 2-3 short owned golden videos and asserts the **number of reels does
not change**
5     vs a frozen baseline. Unlike ``scripts/nightly_regression.py`` (PR #202 ? a CPU re-score
of CACHED
6     trajectories, the windowing/scorer only), this one runs the *full* pipeline on the *real
model*, so it
7     needs a GPU ? hence it runs on an ephemeral GCP GPU VM
(``deploy/gcp/nightly_regression/``), emails the
8     owner the report, and self-deletes. It is **default-additive**: a tripwire, not the
promotion gate.
9
10    What it asserts / reports, per the owner's ask ("quality metrics + regression report +
cost"):
11    ? REEL COUNT per video ? ``len(play_segments)`` after segmentation; the detected
highlight clips.
12    Pure-CV / WASB detection is deterministic for a fixed (video, config, weights), so
this is asserted
13    **EXACT** (with a re-run-once guard for the one known flake: a transient DB-insert-

```

None, see \$guard).

```
14     ? QUALITY METRICS per video (when golden labels are provided) ? the R2 tolerance-match
block from
15     ``backend.eval.rally_seg_eval`` (tol_f1 / f1@0.5 / f1@0.25), compared with a small
tolerance.
16     ? COST of the run ? VM uptime (or processing wall) × machine $/hr via
``backend.billing`` ? USD + INR.
17
18     Extensible pattern (add a video = one manifest row + re-freeze):
19     ``eval_baselines/reelcount_manifest.json`` ? the videos (gs:// / local / gdrive://
URIs) + segmenter
20     ``eval_baselines/reelcount_baseline.json`` ? the frozen expected counts (regenerated,
never hand-edited)
21
22     Usage
23     -----
24     Freeze the baseline from the current configured+checkpointed engine (after an intended
change, or first run):
25         python scripts/nightly_reelcount.py --update-baseline
26
27     Nightly check (exit 1 on regression; writes output/nightly_reelcount/<date>.md; emails
if --email):
28         python scripts/nightly_reelcount.py --email
29
30     Exit codes: 0 pass · 1 regression (reel-count changed / a video errored / quality
dropped) · 2 operational
31     (manifest/videos missing, nothing ran) · 3 no baseline · 4 stale (corpus/segmenter
changed ? re-freeze).
32     """
33     from __future__ import annotations
34
35     import argparse
36     import datetime as _dt
37     import json
38     import os
39     import sys
40     from dataclasses import dataclass, field
41     from typing import Any, Dict, List, Optional, Tuple
42
43     REPO_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
44     if REPO_ROOT not in sys.path:
45         sys.path.insert(0, REPO_ROOT)
46
47     # billing is pure (typing only) ? safe to import at module top.
48     from backend import billing # noqa: E402
49
50     DEFAULT_MANIFEST = os.path.join(REPO_ROOT, "eval_baselines", "reelcount_manifest.json")
51     DEFAULT_BASELINE = os.path.join(REPO_ROOT, "eval_baselines", "reelcount_baseline.json")
52     DEFAULT_REPORT_DIR = os.path.join(REPO_ROOT, "output", "nightly_reelcount")
53
54     #: Quality metrics we report + (when labels exist) gate ? higher is better, small
tolerance.
55     QUALITY_HIGHER: Tuple[str, ...] = ("tol_f1", "f1@0.5", "f1@0.25")
56     DEFAULT_QUALITY_TOL = 0.02 # quality is float/labeled ? a looser tripwire than the
exact reel-count
57     DEFAULT_FX_INR_PER_USD = 83.0
58     #: Fallback machine price if the manifest doesn't pin one (T4 on n1-standard-8, asia-
south1, ~mid-2026).
59     DEFAULT_MACHINE_USD_PER_HR = 0.35
60
61
62     @dataclass(frozen=True)
63     class Tolerances:
64         quality_tol: float = DEFAULT_QUALITY_TOL
65
```

```

66         def to_dict(self) -> Dict[str, Any]:
67             return {"quality_tol": self.quality_tol}
68
69
70         # ----- #
71         # Pure cost math (wraps backend.billing ? no IO).
72         # ----- #
73         def cost_report(billed_seconds: float, machine_usd_per_hr: float,
74                         fx_inr_per_usd: float, gpu_seconds: float = 0.0) -> Dict[str, Any]:
75             """Run cost from billed wall-seconds × machine rate ? USD + INR (``backend.billing``
76 arithmetic).
77             ``billed_seconds`` is the VM uptime (boot?delete) when the orchestrator passes
78             ``--vm-uptime-seconds``;
79             otherwise it's the in-process pipeline wall time (a lower bound ? the VM also pays
80 for boot/install)."""
81             rate_per_sec = float(machine_usd_per_hr) / 3600.0
82             usage = {billing.WALL_SECONDS: float(billed_seconds)}
83             rates = {billing.WALL_SECONDS: rate_per_sec}
84             usd, breakdown = billing.compute_cost_usd(usage, rates)
85             return {
86                 "billed_seconds": round(float(billed_seconds), 1),
87                 "gpu_seconds": round(float(gpu_seconds), 1),
88                 "machine_usd_per_hr": float(machine_usd_per_hr),
89                 "usd": usd,
90                 "inr": billing.to_inr(usd, fx_inr_per_usd),
91                 "breakdown": breakdown,
92             }
93
94         # ----- #
95         # Pure summary + comparison core (no IO ? unit-testable with plain dicts).
96         # ----- #
97         def summarize_results(run_results: List[Dict[str, Any]], segmenter: str,
98                               cost: Dict[str, Any]) -> Dict[str, Any]:
99             """Compact, comparable snapshot from per-video run results.
100             Each run result: ``{name, reel_count, ok, error, quality: {tol_f1,...}|None,
101 wall_seconds}``.
102             A video that errored keeps ``reel_count=None`` + ``ok=False`` (so the diff flags it,
103 never hides it)."""
104             per_video: Dict[str, Any] = {}
105             for r in run_results:
106                 per_video[r["name"]] = {
107                     "reel_count": r.get("reel_count"),
108                     "ok": bool(r.get("ok", False)),
109                     "error": r.get("error"),
110                     "quality": r.get("quality"),
111                 }
112             return {"segmenter": segmenter, "cost": cost, "per_video": per_video,
113                     "n": len(per_video)}
114
115     @dataclass
116     class Finding:
117         video: str
118         metric: str # "reel_count" | "error" | a quality key | "segmenter" |
119 "corpus"
120         kind: str # "regression" | "improvement" | "stale"
121         baseline: Optional[float] = None
122         current: Optional[float] = None
123         detail: str = ""
124
125         def message(self) -> str:
126             if self.kind == "stale":

```

```

125         return f"[STALE] {self.metric}: {self.detail}"
126     if self.metric == "error":
127         return f"[REGRESSION] {self.video}: pipeline ERROR ? {self.detail}"
128     b = "?" if self.baseline is None else f"{self.baseline:g}"
129     c = "?" if self.current is None else f"{self.current:g}"
130     tag = "REGRESSION" if self.kind == "regression" else "improvement"
131     return f"[{tag}] {self.video}/{self.metric}: {b} ? {c}"
132
133
134 @dataclass
135 class NightlyResult:
136     findings: List[Finding] = field(default_factory=list)
137     added: List[str] = field(default_factory=list)
138     removed: List[str] = field(default_factory=list)
139
140     @property
141     def regressions(self) -> List[Finding]:
142         return [f for f in self.findings if f.kind == "regression"]
143
144     @property
145     def improvements(self) -> List[Finding]:
146         return [f for f in self.findings if f.kind == "improvement"]
147
148     @property
149     def stale(self) -> List[Finding]:
150         return [f for f in self.findings if f.kind == "stale"]
151
152     def overall_status(self) -> str:
153         if self.regressions:
154             return "REGRESSION"
155         if self.stale:
156             return "STALE"
157         return "PASS"
158
159     def exit_code(self) -> int:
160         if self.regressions:
161             return 1
162         if self.stale:
163             return 4
164         return 0
165
166
167     def compare(baseline: Dict[str, Any], current: Dict[str, Any], tols: Tolerances) ->
NightlyResult:
168         """Diff current run vs frozen baseline.
169
170         * Segmenter changed ? STALE (numbers not comparable; re-freeze).
171         * Corpus (video set) changed ? STALE + report added/removed; still diff the
INTERSECTION.
172         * Per shared video: reel_count EXACT (any change = regression); a pipeline ERROR =
regression;
173         quality metrics drop beyond ``quality_tol`` = regression (rise = improvement).
174         """
175         res = NightlyResult()
176         if baseline.get("segmenter") != current.get("segmenter"):
177             res.findings.append(Finding("", "segmenter", "stale",
178                                     detail=f"{baseline.get('segmenter')} ?
{current.get('segmenter')}"))
179             return res
180
181         b_pv, c_pv = baseline.get("per_video", {}), current.get("per_video", {})
182         res.added = sorted(set(c_pv) - set(b_pv))
183         res.removed = sorted(set(b_pv) - set(c_pv))
184         if res.added or res.removed:
185             res.findings.append(Finding("", "corpus", "stale",

```

```

186                                     detail=f"added={res.added} removed={res.removed}"))
187
188     for v in sorted(set(b_pv) & set(c_pv)):
189         b, c = b_pv[v], c_pv[v]
190         # 1) pipeline must still succeed
191         if not c.get("ok", False) or c.get("reel_count") is None:
192             res.findings.append(Finding(v, "error", "regression",
193                                     detail=str(c.get("error") or "no reel_count
produced"))))
194             continue
195         # 2) reel count ? EXACT
196         bc, cc = b.get("reel_count"), c.get("reel_count")
197         if bc is not None and cc is not None and cc != bc:
198             res.findings.append(Finding(v, "reel_count", "regression", float(bc),
float(cc)))
199         # 3) quality metrics ? tolerance (only if both sides have them)
200         bq, cq = b.get("quality") or {}, c.get("quality") or {}
201         for m in QUALITY_HIGHER:
202             bv, cv = bq.get(m), cq.get(m)
203             if bv is None or cv is None:
204                 continue
205             if cv < bv - tols.quality_tol:
206                 res.findings.append(Finding(v, m, "regression", bv, cv))
207             elif cv > bv + tols.quality_tol:
208                 res.findings.append(Finding(v, m, "improvement", bv, cv))
209     return res
210
211
212     # ----- #
213     # Report formatting (pure).
214     # ----- #
215     def _q(qd: Optional[Dict[str, Any]], key: str) -> str:
216         if not qd or qd.get(key) is None:
217             return "?"
218         return f"{qd[key]:.3f}"
219
220
221     def format_report(baseline: Dict[str, Any], current: Dict[str, Any],
222                     result: NightlyResult, date: str, head: str) -> str:
223         status = result.overall_status()
224         badge = {"PASS": "? PASS", "REGRESSION": "? REGRESSION",
225                 "STALE": "? STALE (re-freeze the baseline)"}[status]
226         cost = current.get("cost", {})
227         L: List[str] = [
228             f"# Nightly engine regression (reel-count + quality + cost) ? {date}",
229             "",
230             f"***Status: {badge}** · exit code {result.exit_code()} · segmenter
`{current.get('segmenter')}`",
231             "",
232             f"- HEAD `{head}` · baseline `{baseline.get('git_commit', '?')}` (frozen
{baseline.get('generated_at', '?')})",
233             f"- **Run cost: ${cost.get('usd', 0):.4f} (?{cost.get('inr', 0):.2f})** · "
234             f"billed {cost.get('billed_seconds', 0):.0f}s @ ${cost.get('machine_usd_per_hr',
0)}/hr",
235             "",
236         ]
237         if result.regressions:
238             L += ["## ? Regressions", ""] + [f"- {f.message()}" for f in result.regressions]
239         if result.stale:
240             L += ["## ? Stale / not comparable", ""] + [f"- {f.message()}" for f in
result.stale]
241         L += ["- Action: `python scripts/nightly_reelcount.py --update-baseline` (after
confirming the change is intended).", ""]
242

```

```

243     # Per-video table.
244     b_pv, c_pv = baseline.get("per_video", {}), current.get("per_video", {})
245     L += ["## Per-video", "",
246           "| video | reels (base?now) | tol_f1 (base?now) | f1@0.5 | status |",
247           "|---|---|---|---|---|"]
248     for v in sorted(set(b_pv) | set(c_pv)):
249         b, c = b_pv.get(v, {}), c_pv.get(v, {})
250         bc = b.get("reel_count")
251         cc = c.get("reel_count")
252         rc = f"'?' if bc is None else bc?{'ERROR' if not c.get('ok', True) else ('?'
if cc is None else cc)}"
253         bq, cq = b.get("quality"), c.get("quality")
254         tf = f"_q(bq, 'tol_f1')?_q(cq, 'tol_f1')?"
255         f5 = f"_q(bq, 'f1@0.5')?_q(cq, 'f1@0.5')?"
256         vstatus = "?" if any(f.video == v and f.kind == "regression" for f in
result.findings) else "?"
257         L.append(f"| {v} | {rc} | {tf} | {f5} | {vstatus} |")
258     L.append("")
259
260     if result.improvements:
261         L += ["_Improvements over baseline (re-freeze to ratchet up):_"] \
262             + [f"- {f.message()}" for f in result.improvements] + [""]
263     L += ["---",
264           "_Full configured+checkpointed pipeline on the real model (GCP GPU VM).
Tripwire only ? does "
265           "not change the promotion gate. Complements PR #202's cached-trajectory CPU
re-score._"]
266     return "\n".join(L)
267
268
269     # ----- #
270     # IO shell ? running the configured pipeline per video (needs backend + GPU + video).
271     # ----- #
272     def _load_gts(labels_ref: Optional[str]) -> Optional[List[Tuple[float, float]]]:
273         """Load ground-truth rally intervals from a golden-features JSON (`gts` key) or a
rallies CSV
274         (`start,end` columns / pairs). Returns None when no labels are configured."""
275         if not labels_ref:
276             return None
277         from backend.storage import fetch
278         local = fetch(labels_ref)
279         if local.endswith(".json"):
280             with open(local, encoding="utf-8") as fh:
281                 data = json.load(fh)
282                 return [(float(s), float(e)) for s, e in (data.get("gts") or [])]
283         # CSV: header start,end (or first two numeric columns)
284         import csv
285         out: List[Tuple[float, float]] = []
286         with open(local, encoding="utf-8") as fh:
287             for row in csv.reader(fh):
288                 try:
289                     out.append((float(row[0]), float(row[1])))
290                 except (ValueError, IndexError):
291                     continue
292         return out
293
294
295     def _segments_to_intervals(segs: List[Dict[str, Any]]) -> List[Tuple[float, float]]:
296         """(start, end)` pairs from segmenter result dicts, tolerant of the two key
conventions in the
297         codebase (`start_time`/`end_time` ? motion/DB shape ? or `start`/`end`).
Used only for the
298         optional quality metrics; the reel COUNT is `len(segs)` regardless, so a key
mismatch degrades
299         quality scoring gracefully (skipped) without ever miscounting reels."""

```



```

300     out: List[Tuple[float, float]] = []
301     for s in segs:
302         start = s.get("start_time", s.get("start"))
303         end = s.get("end_time", s.get("end"))
304         if start is not None and end is not None:
305             out.append((float(start), float(end)))
306     return out
307
308
309     def run_one_video(video: Dict[str, Any], segmenter: str, config: Any,
310                      rerun_on_mismatch: bool = True,
311                      baseline_count: Optional[int] = None) -> Dict[str, Any]:
312         """Run the configured pipeline on one video ? reel count (+ quality vs labels). Lazy
backend imports.
313
314         The re-run-once guard (the one known non-determinism ? a transient
``add_play_segment``?None drop,
315         database.py): if the count != the frozen baseline, re-process the video ONCE; a
transient drop won't
316         reproduce, a real change will."""
317         import random
318         import time
319
320         from backend.eval import rally_seg_eval
321         from backend.infrastructure.database import Database
322         from backend.pipeline.segmenters import segmenter_registry
323         from backend.results import Failure
324         from backend.storage import fetch
325         from backend.utils.hashing import compute_video_id
326
327         name = video["name"]
328         try:
329             local = fetch(video["uri"])
330         except Exception as e: # noqa: BLE001 ? surface, never hide
331             return {"name": name, "reel_count": None, "ok": False, "error": f"fetch failed:
{e}",
332                    "quality": None, "wall_seconds": 0.0}
333
334         gts = _load_gts(video.get("labels_uri"))
335         seg_cls = segmenter_registry.get(segmenter)
336         if not seg_cls:
337             return {"name": name, "reel_count": None, "ok": False,
338                    "error": f"segmenter '{segmenter}' not registered", "quality": None,
"wall_seconds": 0.0}
339
340         def _process(db_path: str) -> Tuple[Optional[int], Optional[List[Tuple[float,
float]]], float, Optional[str]]:
341             random.seed(0) # close motion.py's unseeded metadata RNG so secondary fields
are reproducible too
342             db = Database(db_path)
343             video_id = compute_video_id(local)
344             t0 = time.monotonic()
345             result = seg_cls(db, config).process_video(video_id, local)
346             wall = time.monotonic() - t0
347             if isinstance(result, Failure):
348                 return None, None, wall, getattr(result, "message", "segmenter failed")
349             segs = result.value if hasattr(result, "value") else result
350             intervals = _segments_to_intervals(segs)
351             return len(segs), intervals, wall, None
352
353         import tempfile
354         total_wall = 0.0
355         with tempfile.TemporaryDirectory(prefix="nightly_reel_") as td:
356             count, intervals, wall, err = _process(os.path.join(td, "run1.db"))
357             total_wall += wall

```

```

358         if err is not None:
359             return {"name": name, "reel_count": None, "ok": False, "error": err,
360                    "quality": None, "wall_seconds": round(total_wall, 2)}
361         # re-run-once guard for the transient DB-drop flake
362         if rerun_on_mismatch and baseline_count is not None and count != baseline_count:
363             count2, intervals2, wall2, err2 = _process(os.path.join(td, "run2.db"))
364             total_wall += wall2
365             if err2 is None and count2 is not None:
366                 count, intervals = count2, intervals2 # the reproduced (or corrected)
value
367
368         quality = None
369         if gts and intervals is not None:
370             row = rally_seg_eval.score_intervals(intervals, gts)
371             quality = {k: row[k] for k in ("tol_f1", "f1@0.5", "f1@0.25",
"tol_precision", "tol_recall")}
372                 if k in row}
373
374         return {"name": name, "reel_count": count, "ok": True, "error": None,
375                "quality": quality, "wall_seconds": round(total_wall, 2)}
376
377
378     def load_manifest(path: str) -> Dict[str, Any]:
379         with open(path, encoding="utf-8") as fh:
380             return json.load(fh)
381
382
383     def run_corpus(manifest: Dict[str, Any], baseline: Optional[Dict[str, Any]],
384                  rerun_on_mismatch: bool = True) -> Tuple[List[Dict[str, Any]], float]:
385         """Run every manifest video through the configured pipeline. Returns (run_results,
total_wall_s)."""
386         from backend.config import load_config
387
388         config = load_config()
389         # apply manifest config overrides onto the typed config (dotted keys)
390         for dotted, val in (manifest.get("config_overrides") or {}).items():
391             _set_dotted(config, dotted, val)
392         segmenter = manifest.get("segmenter") or getattr(getattr(config, "indexing", None),
"default_segmenter", "motion")
393         b_pv = (baseline or {}).get("per_video", {})
394         results: List[Dict[str, Any]] = []
395         total_wall = 0.0
396         for v in manifest.get("videos", []):
397             base_count = (b_pv.get(v["name"]) or {}).get("reel_count")
398             r = run_one_video(v, segmenter, config, rerun_on_mismatch, base_count)
399             total_wall += float(r.get("wall_seconds") or 0.0)
400             results.append(r)
401         return results, total_wall
402
403
404     def _set_dotted(config: Any, dotted: str, value: Any) -> None:
405         """Best-effort dotted-path set onto a pydantic config (e.g.
'indexing.motion_threshold')."""
406         parts = dotted.split(".")
407         obj = config
408         for p in parts[:-1]:
409             obj = getattr(obj, p, None)
410             if obj is None:
411                 return
412         try:
413             setattr(obj, parts[-1], value)
414         except Exception: # noqa: BLE001 ? overrides are best-effort; a bad key shouldn't
crash the run
415             pass
416

```

```

417
418 def _git_head() -> str:
419     import subprocess
420     try:
421         out = subprocess.run(["git", "-C", REPO_ROOT, "rev-parse", "--short", "HEAD"],
422                               capture_output=True, text=True, timeout=10)
423         return out.stdout.strip() or "?"
424     except Exception: # noqa: BLE001
425         return "?"
426
427
428 def load_baseline(path: str) -> Optional[Dict[str, Any]]:
429     if not os.path.exists(path):
430         return None
431     with open(path, encoding="utf-8") as fh:
432         return json.load(fh)
433
434
435 def save_baseline(path: str, snapshot: Dict[str, Any]) -> None:
436     out = dict(snapshot)
437     out["generated_at"] = _dt.date.today().isoformat()
438     out["git_commit"] = _git_head()
439     out["_doc"] = ("Frozen nightly reel-count baseline. Regenerate with `python
scripts/nightly_reelcount.py "
440                   "--update-baseline` after an INTENDED engine change or when adding a
video. Tied to the "
441                   "configured+checkpointed engine + the manifest corpus.")
442     out.pop("cost", None) # cost is per-run, not part of the frozen baseline
443     os.makedirs(os.path.dirname(path), exist_ok=True)
444     tmp = path + ".tmp"
445     with open(tmp, "w", encoding="utf-8") as fh:
446         json.dump(out, fh, indent=2, sort_keys=True)
447         fh.write("\n")
448     os.replace(tmp, path)
449
450
451 def build_parser() -> argparse.ArgumentParser:
452     p = argparse.ArgumentParser(description="Nightly E2E engine regression: reel-count +
quality + cost.")
453     p.add_argument("--manifest", default=DEFAULT_MANIFEST)
454     p.add_argument("--baseline", default=DEFAULT_BASELINE)
455     p.add_argument("--report-dir", default=DEFAULT_REPORT_DIR)
456     p.add_argument("--quality-tol", type=float, default=DEFAULT_QUALITY_TOL)
457     p.add_argument("--vm-uptime-seconds", type=float, default=None,
458                   help="authoritative VM uptime (boot?delete) for the cost line; "
459                   "defaults to the in-process pipeline wall time (a lower bound)")
460     p.add_argument("--no-rerun-guard", action="store_true",
461                   help="disable the re-run-once-on-mismatch guard (the DB-drop flake
mitigation)")
462     p.add_argument("--update-baseline", action="store_true",
463                   help="freeze the current reel counts + quality as the new baseline
instead of checking")
464     p.add_argument("--email", action="store_true", help="email the report (see
scripts/nightly_email.py)")
465     p.add_argument("--email-to", default=None, help="override the report recipient")
466     p.add_argument("--date", default=None, help="override the report date (YYYY-MM-DD;
for tests)")
467     p.add_argument("--no-report", action="store_true", help="skip writing the dated
report file")
468     return p
469
470
471 def main(argv: Optional[List[str]] = None) -> int:
472     import logging
473     logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")

```

```

474     log = logging.getLogger("nightly_reelcount")
475     args = build_parser().parse_args(argv)
476     tols = Tolerances(quality_tol=args.quality_tol)
477     date = args.date or _dt.date.today().isoformat()
478
479     if not os.path.exists(args.manifest):
480         log.error("manifest not found: %s", args.manifest)
481         return 2
482     manifest = load_manifest(args.manifest)
483     baseline = load_baseline(args.baseline)
484
485     try:
486         run_results, total_wall = run_corpus(manifest, baseline, rerun_on_mismatch=not
args.no_rerun_guard)
487     except Exception as e: # noqa: BLE001 ? operational failure (deps/config) ? exit 2,
never a false "pass"
488         log.error("run failed: %s", e)
489         return 2
490     if not run_results:
491         log.error("no videos in manifest %s", args.manifest)
492         return 2
493
494     billed = args.vm_uptime_seconds if args.vm_uptime_seconds is not None else
total_wall
495     cost = cost_report(
496         billed_seconds=billed,
497         machine_usd_per_hr=float(manifest.get("machine_usd_per_hr",
DEFAULT_MACHINE_USD_PER_HR)),
498         fx_inr_per_usd=float(manifest.get("fx_inr_per_usd", DEFAULT_FX_INR_PER_USD)),
499         gpu_seconds=total_wall,
500     )
501     segmenter = manifest.get("segmenter", "motion")
502     snapshot = summarize_results(run_results, segmenter, cost)
503     snapshot["git_commit"] = _git_head()
504     snapshot["generated_at"] = date
505
506     if args.update_baseline:
507         save_baseline(args.baseline, snapshot)
508         log.info("baseline frozen ? %s", args.baseline)
509         for r in run_results:
510             log.info(" %s: reels=%s ok=%s", r["name"], r.get("reel_count"),
r.get("ok"))
511         return 0
512
513     if baseline is None:
514         log.error("no baseline at %s ? freeze one with --update-baseline (on the
configured engine).", args.baseline)
515         return 3
516
517     result = compare(baseline, snapshot, tols)
518     report = format_report(baseline, snapshot, result, date, snapshot["git_commit"])
519
520     if not args.no_report:
521         os.makedirs(args.report_dir, exist_ok=True)
522         md_path = os.path.join(args.report_dir, f"{date}.md")
523         with open(md_path, "w", encoding="utf-8") as fh:
524             fh.write(report + "\n")
525         with open(os.path.join(args.report_dir, f"{date}.json"), "w", encoding="utf-8")
as fh:
526             json.dump({"date": date, "status": result.overall_status(), "exit_code":
result.exit_code(),
527                       "snapshot": snapshot, "regressions": [f.message() for f in
result.regressions]},
528                     fh, indent=2, sort_keys=True)
529         log.info("report ? %s", md_path)

```

```

530
531     print(report)
532
533     if args.email:
534         try:
535             from scripts.nightly_email import send_report
536             subject = f"[KheISutra nightly] {result.overall_status()} ? reel-count
engine regression {date}"
537             sent = send_report(subject, report, to=args.email_to)
538             log.info("email %s", "sent" if sent else "skipped (no creds ? report still
written)")
539             except Exception as e: # noqa: BLE001 ? a broken mailer must NOT mask the
regression verdict
540                 log.error("email failed (report still written): %s", e)
541
542         code = result.exit_code()
543         if code == 1:
544             log.error("REEL-COUNT REGRESSION (%d) ? see %s", len(result.regressions),
args.report_dir)
545         elif code == 4:
546             log.warning("baseline STALE ? corpus/segmenter changed; re-freeze with --update-
baseline")
547         else:
548             log.info("nightly reel-count check PASSED (cost $%.4f / ?%.2f)", cost["usd"],
cost["inr"])
549         return code
550
551
552     if __name__ == "__main__":
553         raise SystemExit(main())
554

```

5. user (2026-06-22T09:45:40.288Z)

```

1     #!/usr/bin/env bash
2     # Startup-script for the NIGHTLY engine-regression VM. Runs UNATTENDED on boot, then
DELETES the VM
3     # (cost rule: a running GPU VM is the only real cost). Survives SSH drops (it's a
startup-script, not
4     # an interactive session). Reads its config from instance metadata (set by launch.sh).
5     #
6     # Flow: Ops Agent ? pull repo @ HEAD from GCS ? build venv + WASB env ? stage
weights ?
7     #         run scripts/nightly_reelcount.py on the gs:// golden clips (emails the report)
?
8     #         upload report to GCS ? SELF-DELETE (even on failure, via the EXIT trap).
9     #
10    # Metadata keys (launch.sh sets these): repo-tgz-uri, git-sha, gcs-bucket, report-
prefix,
11    # weights-uri, email (true/false). NOT verified in CI ? dry-run via launch.sh on a
real VM first.
12    set -uo pipefail
13    exec >>(tee -a /var/log/nightly-regression.log) 2>&1
14    echo "==== nightly-regression startup $(date -u +%FT%TZ) ====="
15    BOOT_EPOCH=$(date +%s)
16
17    md() { curl -s -H "Metadata-Flavor: Google"
"http://metadata.google.internal/computeMetadata/v1/$1"; }
18    NAME=$(md instance/name)
19    ZONE=$(md instance/zone | awk -F/ '{print $NF}')
20    REPO_TGZ=$(md instance/attributes/repo-tgz-uri || true)
21    GIT_SHA=$(md instance/attributes/git-sha || echo unknown)
22    BUCKET=$(md instance/attributes/gcs-bucket || true)
23    REPORT_PREFIX=$(md instance/attributes/report-prefix || echo nightly/reports)
24    WEIGHTS_URI=$(md instance/attributes/weights-uri || true)

```

```

25     DO_EMAIL=$(md instance/attributes/email || echo true)
26     SMTP_SECRET=$(md instance/attributes/smtp-secret || true)
27     SMTP_USER=$(md instance/attributes/smtp-user || true)
28     EMAIL_TO=$(md instance/attributes/email-to || echo avi.dullu@gmail.com)
29     DATE=$(date -u +%F)
30
31     # ALWAYS delete the VM on exit (success OR failure) ? never leave a billing GPU VM
behind.
32     cleanup() {
33         echo "==== self-delete $NAME ($ZONE) ====="
34         gcloud compute instances delete "$NAME" --zone="$ZONE" --quiet || \
35             echo "WARN: self-delete failed ? DELETE MANUALLY: gcloud compute instances delete
$NAME --zone=$ZONE -q"
36     }
37     trap cleanup EXIT
38
39     fail_email() { # best-effort failure alert (the report email won't have fired on an
early crash)
40         echo "RUN FAILED ? attempting a failure alert"
41         if [ "$DO_EMAIL" = "true" ] && [ -d ~/rally ]; then
42             cd ~/rally 2>/dev/null && . .venv/bin/activate 2>/dev/null && \
43                 NIGHTLY SMTP_SECRET="$SMTP_SECRET" NIGHTLY SMTP_USER="$SMTP_USER"
NIGHTLY_EMAIL_TO="$EMAIL_TO" \
44                 python - <<PY || true
45         from scripts.nightly_email import send_report
46         send_report("[KhelSutra nightly] ERROR ? engine regression VM run failed ($DATE)",
47             "The nightly engine-regression run failed on the VM before producing a
report.\n"
48             "See gs://$BUCKET/$REPORT_PREFIX/$DATE/ and the VM serial
log.\nGIT_SHA=$GIT_SHA", to="$EMAIL_TO")
49         PY
50         fi
51     }
52     trap 'fail_email' ERR
53
54     # 1) Observability (Ops Agent) ? same as create_gpu_vm.sh bakes in.
55     curl -sSO https://dl.google.com/cloudagents/add-google-cloud-ops-agent-repo.sh && \
56         sudo bash add-google-cloud-ops-agent-repo.sh --also-install || echo "WARN: ops-agent
install failed (non-fatal)"
57
58     # 2) Pull the repo @ HEAD (staged to GCS by launch.sh ? no GitHub auth needed; uses the
worker SA).
59     mkdir -p ~/rally && cd ~/rally
60     gcloud storage cp "$REPO_TGZ" /tmp/rally.tgz
61     tar -xzf /tmp/rally.tgz -C ~/rally
62     echo "$GIT_SHA" > ~/rally/GIT_SHA
63
64     # 3) Build the env + native WASB (provider-agnostic DO scripts, reused on GCP per
deploy/gcp/README $4).
65     sudo bash deploy/digitalocean/mount_scratch.sh 2>/dev/null || true
66     export TMPDIR=/scratch 2>/dev/null || true
67     ./deploy/digitalocean/bootstrap.sh --gpu
68     . .venv/bin/activate
69     pip install -e '[storage-gcs]'
70     bash deploy/digitalocean/gpu_setup.sh || echo "WARN: gpu_setup (WASB env) returned
nonzero ? continuing"
71
72     # 4) Stage the WASB checkpoint (gpu_setup does NOT fetch weights ? README $4).
73     if [ -n "$WEIGHTS_URI" ]; then
74         mkdir -p ~/models/WASB-SBDT/pretrained_weights
75         gcloud storage cp "$WEIGHTS_URI" ~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar
76         export WASB_WEIGHTS=~/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar
77     fi
78

```

```

79      # 5) Run the regression on the gs:// golden clips in the committed manifest. Cost = VM
uptime so far.
80      UPTIME=$(( $(date +%s) - BOOT_EPOCH ))
81      EMAIL_FLAG=""; [ "$DO_EMAIL" = "true" ] && EMAIL_FLAG="--email --email-to $EMAIL_TO"
82      set +e
83      NIGHTLY_SMTP_SECRET="$SMTP_SECRET" NIGHTLY_SMTP_USER="$SMTP_USER"
NIGHTLY_EMAIL_TO="$EMAIL_TO" \
84      python scripts/nightly_reelcount.py --vm-uptime-seconds "$UPTIME" $EMAIL_FLAG
85      RUN_RC=$?
86      set -e
87      echo "runner exit code: $RUN_RC (0 pass / 1 regression / 4 stale)"
88
89      # 6) Upload the dated report to GCS for the record (alongside the email).
90      if [ -n "$BUCKET" ] && [ -d output/nightly_reelcount ]; then
91          gcloud storage cp -r "output/nightly_reelcount/$DATE."*
"gs://$BUCKET/$REPORT_PREFIX/$DATE/" || \
92          gcloud storage cp -r output/nightly_reelcount "gs://$BUCKET/$REPORT_PREFIX/$DATE/"
|| true
93      fi
94
95      echo "==== nightly-regression done rc=$RUN_RC $(date -u +%FT%TZ) ====="
96      trap - ERR      # disable the failure-email trap; the EXIT (self-delete) trap stays
97      # the EXIT trap deletes the VM
98

```

6. assistant (2026-06-22T09:45:43.225Z)

Now let me read the remaining deployment and manifest files to complete the analysis:

7. user (2026-06-22T09:45:44.273Z)

```

1      #!/usr/bin/env bash
2      # Launch ONE nightly engine-regression run on an ephemeral GCP GPU VM, then return. The
VM runs
3      # run_on_vm.sh as its startup-script (regression ? email ? SELF-DELETE), so this just
stages the repo
4      # and kicks the VM off. Driver-agnostic: call it from Cloud Scheduler?Cloud Run job, a
cron on any
5      # always-on box, the owner's machine, or a Claude routine. Needs gcloud auth + the repo
checkout.
6      #
7      #   bash deploy/gcp/nightly_regression/launch.sh                                # L4, stages HEAD,
kicks the VM
8      #   RALLY_GPU=t4 RALLY_REF=$(git rev-parse HEAD) bash
deploy/gcp/nightly_regression/launch.sh
9      #
10     # ? Run this MANUALLY once as a dry-run (watch the serial log) before wiring a scheduler
? the VM-side
11     # flow is NOT covered by CI. Tail: gcloud compute instances get-serial-port-output
<name> --zone=<z>.
12     set -uo pipefail
13
14     PROJECT="${RALLY_GCP_PROJECT:-gen-lang-client-0306026826}"
15     BUCKET="${RALLY_BUCKET:-khelsutra-rally-corpus}"
16     SA="rally-worker@${PROJECT}.iam.gserviceaccount.com"
17     GPU="${RALLY_GPU:-l4}"
18     NAME="${RALLY_VM_NAME:-rally-nightly-$(date -u +%Y%m%d)}"
19     REF="${RALLY_REF:-HEAD}"
20     WEIGHTS_URI="${RALLY_WEIGHTS_URI:-gs://$BUCKET/weights/wasb_badminton_best.pth.tar}"
21     REPORT_PREFIX="${RALLY_REPORT_PREFIX:-nightly/reports}"
22     DO_EMAIL="${RALLY_EMAIL:-true}"
23     SMTP_SECRET="${NIGHTLY_SMTP_SECRET:-projects/$PROJECT/secrets/nightly-smtp-
pass/versions/latest}"
24     SMTP_USER="${NIGHTLY_SMTP_USER:-}"
25     EMAIL_TO="${NIGHTLY_EMAIL_TO:-avi.dullu@gmail.com}"

```

```

26     HERE="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
27     STARTUP="$HERE/run_on_vm.sh"
28     export CLOUDSDK_CORE_DISABLE_PROMPTS=1
29
30     ZONES="${RALLY_GCP_ZONES:-asia-south1-a asia-south1-b asia-south1-c asia-southeast1-a
asia-southeast1-b asia-southeast1-c}"
31     if [ "$GPU" = "l4" ]; then MACHINE="g2-standard-4"; ACCEL=(); else
MACHINE="n1-standard-8"; ACCEL=(--accelerator=type=nvidia-tesla-t4,count=1); fi
32
33     # 1) Stage the repo @ REF to GCS (no GitHub auth on the VM; the worker SA can read the
bucket).
34     SHA="$(git rev-parse "$REF")"
35     TGZ_URI="gs://$BUCKET/nightly/repo/${SHA}.tgz"
36     echo "== staging repo $SHA -> $TGZ_URI =="
37     TMP_TGZ="$(mktemp --suffix=.tgz)"
38     git archive --format=tar.gz -o "$TMP_TGZ" "$REF"
39     gcloud storage cp "$TMP_TGZ" "$TGZ_URI" --project="$PROJECT"
40     rm -f "$TMP_TGZ"
41
42     # 2) Zone-sweep create the VM (mirrors create_gpu_vm.sh) with run_on_vm.sh + the config
metadata.
43     ERR="$(mktemp)"
44     for Z in $ZONES; do
45         echo "== creating $NAME ($GPU) in $Z =="
46         if gcloud compute instances create "$NAME" --project="$PROJECT" --zone="$Z" \
47             --machine-type="$MACHINE" "${ACCEL[@]}" \
48             --maintenance-policy=TERMINATE --provisioning-model=STANDARD \
49             --image-family=common-cu129-ubuntu-2204-nvidia-580 --image-project=deeplearning-
platform-release \
50             --boot-disk-size=120GB --boot-disk-type=pd-balanced \
51             --service-account="$SA" --scopes=cloud-platform \
52             --metadata-from-file=startup-script="$STARTUP" \
53             --metadata=repo-tgz-uri="$TGZ_URI",git-sha="$SHA",gcs-bucket="$BUCKET",report-
prefix="$REPORT_PREFIX",weights-uri="$WEIGHTS_URI",email="$DO_EMAIL",smtp-
secret="$SMTP_SECRET",smtp-user="$SMTP_USER",email-to="$EMAIL_TO" \
54             2>"$ERR"; then
55             echo "? launched $NAME in $Z ? it will run the regression, email $EMAIL_TO, and
self-delete."
56             echo "    serial log: gcloud compute instances get-serial-port-output $NAME --zone=$Z
--project=$PROJECT"
57             echo "    if it doesn't self-delete: gcloud compute instances delete $NAME --zone=$Z
-q"
58             rm -f "$ERR"; exit 0
59         fi
60         if grep -qiE "STOCKOUT|does not have enough" "$ERR"; then echo "    (no $GPU capacity in
$Z ? next zone)"; continue; fi
61         echo "    create failed:"; tail -4 "$ERR"; rm -f "$ERR"; exit 1
62     done
63     echo "? no $GPU capacity in any swept zone ($ZONES). Retry later or RALLY_GPU=t4."
64     rm -f "$ERR"; exit 1
65

```

8. user (2026-06-22T09:45:45.238Z)

```

1     # Tiny launcher image for the nightly engine-regression Cloud Run JOB. It is NOT the GPU
workload ?
2     # it only stages the repo + creates the ephemeral GPU VM (which self-deletes). Scale-to-
zero: the job
3     # runs for a few seconds nightly, then exits. Base = google/cloud-sdk (gcloud + git
preinstalled).
4     FROM gcr.io/google.com/cloudsdktool/cloud-sdk:slim
5     WORKDIR /app
6     # The repo is the build context (run `gcloud builds submit` from the repo root, see
register_scheduler.sh).
7     COPY . /app

```



```
8     ENTRYPOINT ["bash", "deploy/gcp/nightly_regression/launch.sh"]
9
```

9. user (2026-06-22T09:45:45.975Z)

```
1  {
2    "_doc": "Nightly E2E engine reel-count manifest (scripts/nightly_reelcount.py). The
configured+checkpointed pipeline runs on each video on a GCP GPU VM; the reel count + quality
metrics are asserted vs eval_baselines/reelcount_baseline.json. ADD A VIDEO = append one row +
re-freeze: `python scripts/nightly_reelcount.py --update-baseline` (commit the updated
baseline). URIs may be gs:// (recommended, co-located with the VM), gdrive://, or local paths.
labels_uri (optional) is a golden-features JSON (gts key) or a start,end CSV ? present =>
quality metrics (tol_f1/f1@0.5/f1@0.25) are scored too. EDIT the example rows below to your
actual short (~5-6 min) OWNED, minors-free clips before enabling.",
3    "segmenter": "wasb_hybrid",
4    "detector_impl": "native",
5    "config_overrides": {
6      "indexing.default_segmenter": "wasb_hybrid",
7      "indexing.detector_impl": "native"
8    },
9    "machine_usd_per_hr": 0.70,
10   "fx_inr_per_usd": 83.0,
11   "videos": [
12     {
13       "name": "GX030094",
14       "uri": "gs://khehsutra-rally-corpus/nightly/videos/GX030094.mp4",
15       "labels_uri": "gs://khehsutra-rally-
corpus/nightly/labels/GX030094_golden_features.json"
16     },
17     {
18       "name": "Badminton_BXH_2",
19       "uri": "gs://khehsutra-rally-corpus/nightly/videos/Badminton_BXH_2.mp4",
20       "labels_uri": "gs://khehsutra-rally-
corpus/nightly/labels/Badminton_BXH_2_golden_features.json"
21     },
22     {
23       "name": "GX010128",
24       "uri": "gs://khehsutra-rally-corpus/nightly/videos/GX010128.mp4",
25       "labels_uri": "gs://khehsutra-rally-
corpus/nightly/labels/GX010128_golden_features.json"
26     }
27   ]
28 }
29
```

10. assistant (2026-06-22T09:45:48.305Z)

Now let me check the register_scheduler.sh file:

11. user (2026-06-22T09:45:49.388Z)

```
1  #!/usr/bin/env bash
2  # Wire the nightly engine-regression as a CLOUD-NATIVE, scale-to-zero job:
3  #   Cloud Scheduler (cron) ? Cloud Run JOB (the launcher image) ? launch.sh
4  #   ? ephemeral GPU VM (regression + email + SELF-DELETE).
5  # The only always-on cost is $0 (Cloud Run jobs + Scheduler are scale-to-zero; the GPU
VM is ephemeral).
6  #
7  # ? OWNER-RUN, ONCE, after a successful manual `bash
deploy/gcp/nightly_regression/launch.sh` dry-run.
8  # NOT verified in CI (no GCP/GPU here). Prereqs: GPU quota (README §2), the golden clips
+ weights in
9  # gs://<bucket>, and the SMTP secret (see README "Enable").
10 #
```

```

11      #   bash deploy/gcp/nightly_regression/register_scheduler.sh           # build + deploy
+ schedule (03:00 IST)
12      #   RALLY_CRON='0 3 * * *' RALLY_TZ='Asia/Kolkata' bash ../register_scheduler.sh
13      set -euo pipefail
14
15      PROJECT="${RALLY_GCP_PROJECT:-gen-lang-client-0306026826}"
16      REGION="${RALLY_RUN_REGION:-asia-south1}"
17      JOB="${RALLY_RUN_JOB:-nightly-regression-launcher}"
18      SCHED="${RALLY_SCHED_NAME:-nightly-regression}"
19      CRON="${RALLY_CRON:-0 3 * * *}"           # 03:00 daily
20      TZ="${RALLY_TZ:-Asia/Kolkata}"
21      SA="rally-worker@${PROJECT}.iam.gserviceaccount.com"
22      IMAGE="${RALLY_LAUNCHER_IMAGE:-${REGION}-docker.pkg.dev/${PROJECT}/rally/nightly-
launcher:latest}"
23      REPO_ROOT="$(git rev-parse --show-toplevel)"
24      export CLOUDSDK_CORE_DISABLE_PROMPTS=1
25
26      echo "== enable APIs (run, scheduler, build, artifact registry) =="
27      gcloud services enable run.googleapis.com cloudscheduler.googleapis.com \
28      cloudbuild.googleapis.com artifactregistry.googleapis.com --project="$PROJECT"
29
30      echo "== ensure the Artifact Registry repo 'rally' exists =="
31      gcloud artifacts repositories describe rally --location="$REGION" --project="$PROJECT"
>/dev/null 2>&1 || \
32      gcloud artifacts repositories create rally --repository-format=docker
--location="$REGION" --project="$PROJECT"
33
34      echo "== build + push the launcher image from the repo root =="
35      ( cd "$REPO_ROOT" && gcloud builds submit --project="$PROJECT" --tag "$IMAGE" \
36      --gcs-source-staging-dir="gs://${RALLY_BUILD_BUCKET:-kshelsutra-rally-
corpus}/cloudbuild" \
37      -f deploy/gcp/nightly_regression/Dockerfile.launcher . )
38
39      echo "== create/update the Cloud Run JOB (launcher needs compute-admin to create the GPU
VM) =="
40      gcloud run jobs deploy "$JOB" --project="$PROJECT" --region="$REGION" \
41      --image="$IMAGE" --service-account="$SA" --max-retries=0 --task-timeout=900s \
42      --set-env-vars="RALLY_GPU=${RALLY_GPU:-
l4},RALLY_EMAIL=true,NIGHTLY_EMAIL_TO=${NIGHTLY_EMAIL_TO:-avi.dullu@gmail.com}"
43
44      echo "== schedule it (Cloud Scheduler ? Run job admin API) =="
45      RUN_URI="https://${REGION}-
run.googleapis.com/apis/run.googleapis.com/v1/namespaces/${PROJECT}/jobs/${JOB}:run"
46      gcloud scheduler jobs describe "$SCHED" --location="$REGION" --project="$PROJECT"
>/dev/null 2>&1 && \
47      gcloud scheduler jobs delete "$SCHED" --location="$REGION" --project="$PROJECT"
--quiet || true
48      gcloud scheduler jobs create http "$SCHED" --project="$PROJECT" --location="$REGION" \
49      --schedule="$CRON" --time-zone="$TZ" \
50      --uri="$RUN_URI" --http-method=POST \
51      --oauth-service-account-email="$SA"
52
53      cat <<EOF
54
55      ? Scheduled '$SCHED' ($CRON $TZ) ? Cloud Run job '$JOB' ? launch.sh ? ephemeral GPU VM.
56      IAM the launcher SA needs (grant once if missing):
57      roles/compute.instanceAdmin.v1 (create/delete the GPU VM)
58      roles/iam.serviceAccountUser (attach $SA to the VM)
59      roles/run.invoker + roles/cloudscheduler.jobRunner (Scheduler ? Run)
60      Trigger now to test: gcloud run jobs execute $JOB --region=$REGION
--project=$PROJECT
61      Remove: gcloud scheduler jobs delete $SCHED --location=$REGION; gcloud
run jobs delete $JOB --region=$REGION
62      EOF
63

```

12. assistant (2026-06-22T09:45:51.770Z)

Now let me verify the exact SSL/TLS vulnerability claim by examining the SMTP code more closely:

13. user (2026-06-22T09:45:53.652Z)

```
86: with smtplib.SMTP(host, port, timeout=60) as s:
87:     s.starttls()
```

14. assistant (2026-06-22T09:45:55.826Z)

Let me check the imports to see if ssl is used anywhere:

15. user (2026-06-22T09:45:56.668Z)

Exit code 2

```
grep: C:\Users\avidu\Projects\badminton-highlight-indexer\claudeworktrees\nightly-e2escripts\nightly_email.py: No such file or directory
```

16. user (2026-06-22T09:45:59.655Z)

```
1      """Email the nightly regression report ? stdlib SMTP, creds from env or GCP Secret
Manager.
2
3      Used by ``scripts/nightly_reelcount.py --email`` (and reusable by any nightly).
Deliberately
4      dependency-light (stdlib ``smtplib`` + ``email.message``) and graceful: if no SMTP
password can
5      be resolved it logs + returns ``False`` (the report is still written/printed ? a broken
mailer must
6      never mask a regression).
7
8      Credential resolution (first hit wins), so it works on a GCP VM, locally, or in tests:
9      ? Recipient: ``--email-to`` arg ? ``NIGHTLY_EMAIL_TO`` ? ``avi.dullu@gmail.com``
10     ? SMTP host: ``NIGHTLY_SMTP_HOST`` (default ``smtp.gmail.com``) . port
``NIGHTLY_SMTP_PORT`` (587, STARTTLS)
11     ? SMTP user: ``NIGHTLY_SMTP_USER`` (default = recipient)
12     ? SMTP pass: ``NIGHTLY_SMTP_PASS`` ? Secret Manager ``NIGHTLY_SMTP_SECRET``
13     (`projects/<p>/secrets/<name>/versions/latest`; via the google-cloud-
secret-manager
14     lib if installed, else the ``gcloud`` CLI). A Gmail app password is
the simplest.
15
16     Set the secret once (owner): echo -n '<app-password>' | gcloud secrets create nightly-
smtp-pass --data-file=-
17     and on the VM: export NIGHTLY_SMTP_SECRET=projects/<proj>/secrets/nightly-smtp-
pass/versions/latest
18     export NIGHTLY_SMTP_USER=<your-gmail>
NIGHTLY_EMAIL_TO=avi.dullu@gmail.com
19     """
20     from __future__ import annotations
21
22     import logging
23     import os
24     from email.message import EmailMessage
25     from typing import Optional
26
27     log = logging.getLogger("nightly_email")
28
29     DEFAULT_TO = "avi.dullu@gmail.com"
30     DEFAULT_HOST = "smtp.gmail.com"
```

17. assistant (2026-06-22T09:46:01.997Z)

Now let me check Python's smtplib documentation behavior and review if there's any real risk here. Let me search for Python best practices in this codebase:

18. user (2026-06-22T09:46:03.458Z)

(Bash completed with no output)

19. assistant (2026-06-22T09:46:05.486Z)

Let me check if there are any other SMTP connections in the codebase:

20. user (2026-06-22T09:46:06.402Z)

```
./scripts/nightly_email.py:    with smtplib.SMTP(host, port, timeout=60) as s:
```

21. assistant (2026-06-22T09:46:09.237Z)

Now let me verify the actual Python SMTP behavior by checking the Python documentation assertion. Let me analyze the claim adversarially:

22. user (2026-06-22T09:46:10.333Z)

Help on function `__init__` in module `smtplib`:

```
__init__(
    self,
    host='',
    port=0,
    local_hostname=None,
    timeout=<object object at 0x00000243842C0360>,
    source_address=None
)
```

Initialize a new instance.

If specified, `'host'` is the name of the remote host to which to connect. If specified, `'port'` specifies the port to which to connect. By default, `smtplib.SMTP_PORT` is used. If a host is specified the connect method is called, and if it returns anything other than a success code an `SMTPConnectError` is raised. If specified, `'local_hostname'` is used as the FQDN of the local host in the HELO/EHLO command. Otherwise, the local hostname is found using `socket.getfqdn()`. The `'source_address'` parameter takes a 2-tuple (host, port) for the socket to bind to as its source address before connecting. If the host is `''` and port is `0`, the OS default behavior will be used.

23. user (2026-06-22T09:46:13.474Z)

Help on function `starttls` in module `smtplib`:

```
starttls(self, *, context=None)
```

Puts the connection to the SMTP server into TLS mode.

If there has been no previous EHLO or HELO command this session, this method tries ESMTP EHLO first.

If the server supports TLS, this will encrypt the rest of the SMTP session. If you provide the context parameter, the identity of the SMTP server and client can be checked. This, however, depends on whether the socket module really checks the certificates.

This method may raise the following exceptions:

```
SMTPHeloError          The server didn't reply properly to
                        the helo greeting.
```

24. user (2026-06-22T09:46:19.032Z)

```
=== SMTP.starttls source ===
def starttls(self, *, context=None):
    """Puts the connection to the SMTP server into TLS mode.

    If there has been no previous EHLO or HELO command this session, this
    method tries ESMTP EHLO first.

    If the server supports TLS, this will encrypt the rest of the SMTP
    session. If you provide the context parameter,
    the identity of the SMTP server and client can be checked. This,
    however, depends on whether the socket module really checks the
    certificates.

    This method may raise the following exceptions:

    SMTPHeloError          The server didn't reply properly to
                            the helo greeting.
    """
    self.ehlo_or_helo_if_needed()
    if not self.has_extn("starttls"):
        raise SMTPNotSupportedError(
            "STARTTLS extension not supported by server.")
    (resp, reply) = self.docmd("STARTTLS")
    if resp == 220:
        if not _have_ssl:
            raise RuntimeError("No SSL support included in this Python")
        if context is None:
            context = ssl._create_stdlib_context()
        self.sock = context.wrap_socket(self.sock,
                                       server_hostname=self._host)

        self.file = None
        # RFC 3207:
        # The client MUST discard any knowledge obtained from
        # the server, such as the list of SMTP service extensions,
        # which

=== SMTP_SSL (for comparison) ===
def __init__(self, host='', port=0, local_hostname=None,
             *, timeout=socket._GLOBAL_DEFAULT_TIMEOUT,
             source_address=None, context=None):
    if context is None:
        context = ssl._create_stdlib_context()
    self.context = context
    SMTP.__init__(self, host, port, local_hostname, timeout,
                  source_address)

=== Testing default context behavior ===
Default context check_hostname: True
Default context verify_mode: 2
Verify mode is CERT_REQUIRED: True
```

25. assistant (2026-06-22T09:46:21.104Z)

Now I have the critical information. Let me verify what `ssl.\_create\_stdlib\_context()` does:

26. user (2026-06-22T09:46:22.256Z)

```
=== ssl._create_stdlib_context documentation ===
```

Help on function _create_unverified_context in module ssl:

```
_create_unverified_context(
    protocol=None,
    *,
    cert_reqs=<VerifyMode.CERT_NONE: 0>,
    check_hostname=False,
    purpose=<Purpose.SERVER_AUTH: _ASN1Object(nid=129, shortname='serverAuth', longname='TLS Web
Server Authentication', oid='1.3.6.1.5.5.7.3.1')>,
    certfile=None,
    keyfile=None,
    cafile=None,
    capath=None,
    cadata=None
)
```

Create a SSLContext object for Python stdlib modules

All Python stdlib modules shall use this function to create SSLContext objects in order to keep common settings in one place. The configuration is less restrict than create_default_context()'s to increase backward compatibility.

27. assistant (2026-06-22T09:46:24.160Z)

This is the smoking gun! Let me verify the actual defaults:

28. user (2026-06-22T09:46:28.258Z)

```
=== stdlib context (used by smtplib.SMTP.starttls when no context provided) ===
check_hostname: False
verify_mode: 0 (CERT_NONE)
```

```
=== default context (recommended best practice) ===
check_hostname: True
verify_mode: 2 (CERT_REQUIRED)
```

DIFFERENCE: stdlib context does NOT verify certificates by default!
VerifyMode.CERT_NONE (0) = no verification
VerifyMode.CERT_REQUIRED (2) = certificate verification required

29. assistant (2026-06-22T09:46:30.209Z)

Perfect. Now I have conclusive evidence. Let me document the exact vulnerability:

30. user (2026-06-22T09:46:43.915Z)

Structured output provided successfully